

LOGISTIC REGRESSION PIPE FAILURE PREDICTION MODEL

September 12, 2026

```
[2]: import pandas as pd
data = pd.read_csv(r"C:\Users\User\Downloads\DATA ANALYSIS PORTFOLIO\5 WATER_
↳PROJECTS\04_pipe_failure_prediction.csv")
```

```
[4]: data.head(3)
```

```
[4]:   pipe_id  diameter_mm material  installation_year  age_years  length_m  \
0  P-00001          63     PVC             2016         10    1328.6
1  P-00002          50      GI             2011         15     690.1
2  P-00003          63    HDPE             2013         13    2073.1

   operating_pressure_m  soil_type  elevation_m  previous_failures  \
0                39.0     Clay      1478.0             0
1                11.1    Rocky      1768.3             0
2                34.2    Sandy       809.2             1

   leakage_incidents  annual_rainfall_mm  recent_rainfall_mm  \
0                 3             672.4             0.6
1                 1             956.3             6.9
2                 1            1331.9             7.6

   maintenance_count_24m  failure_within_90_days  failure_risk_score  \
0                     4                      1             12.7
1                     3                      0              8.1
2                     3                      0             14.4

   risk_category
0             Low
1             Low
2             Low
```

```
[5]: data.shape
# 6000 rows and 17 columns
```

```
[5]: (6000, 17)
```

```
[8]: data.columns.to_list()
      # all the 17 columns
```

```
[8]: ['pipe_id',
      'diameter_mm',
      'material',
      'installation_year',
      'age_years',
      'length_m',
      'operating_pressure_m',
      'soil_type',
      'elevation_m',
      'previous_failures',
      'leakage_incidents',
      'annual_rainfall_mm',
      'recent_rainfall_mm',
      'maintenance_count_24m',
      'failure_within_90_days',
      'failure_risk_score',
      'risk_category']
```

```
[9]: data.info()
      # gives us the non_null count and datatype of the columns
      #there are no null/missing values in every column
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6000 entries, 0 to 5999
Data columns (total 17 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   pipe_id                               6000 non-null   object
1   diameter_mm                           6000 non-null   int64
2   material                               6000 non-null   object
3   installation_year                      6000 non-null   int64
4   age_years                             6000 non-null   int64
5   length_m                              6000 non-null   float64
6   operating_pressure_m                  6000 non-null   float64
7   soil_type                             6000 non-null   object
8   elevation_m                           6000 non-null   float64
9   previous_failures                     6000 non-null   int64
10  leakage_incidents                     6000 non-null   int64
11  annual_rainfall_mm                    6000 non-null   float64
12  recent_rainfall_mm                    6000 non-null   float64
13  maintenance_count_24m                 6000 non-null   int64
14  failure_within_90_days                 6000 non-null   int64
15  failure_risk_score                     6000 non-null   float64
16  risk_category                          6000 non-null   object
dtypes: float64(6), int64(7), object(4)
```

memory usage: 797.0+ KB

```
[10]: data.isnull().sum()  
      # no missing values
```

```
[10]: pipe_id          0  
      diameter_mm      0  
      material         0  
      installation_year 0  
      age_years        0  
      length_m         0  
      operating_pressure_m 0  
      soil_type        0  
      elevation_m      0  
      previous_failures 0  
      leakage_incidents 0  
      annual_rainfall_mm 0  
      recent_rainfall_mm 0  
      maintenance_count_24m 0  
      failure_within_90_days 0  
      failure_risk_score 0  
      risk_category     0  
      dtype: int64
```

```
[11]: #Our target is failure_within_90_days  
      data['failure_within_90_days'].value_counts()  
  
      # let numeral 1 represent = failure. It was recorded 1574 times  
      # let numeral 0 represent = no failure. It was recorded 4426 times
```

```
[11]: failure_within_90_days  
      0    4426  
      1    1574  
      Name: count, dtype: int64
```

```
[12]: data['failure_within_90_days'].value_counts(normalize= True)*100  
      # percentage proportions of failure and no failure(0) records
```

```
[12]: failure_within_90_days  
      0    73.766667  
      1    26.233333  
      Name: proportion, dtype: float64
```

```
[14]: # IDENTIFY/GET OUR FEATURES(x) AND TARGET(y)  
      #our features are all the columns except failure_within_90_days, pipe_id(it  
      ↪ does not tell Logistic Regression anything meaningful)  
      # we will also remove failure_risk_score and risk_category because they sound  
      ↪ like pre-existing risk assessments derived from other pipe variables.
```

```
# If they were calculated using the same failure outcome we are trying to
↪predict, using them would create data leakage

x = data.
    ↪drop(['failure_risk_score', 'risk_category', 'pipe_id', 'failure_within_90_days'],
    ↪axis=1)
y = data['failure_within_90_days']
```

```
[15]: print(x.shape)
      print(y.shape)
```

```
(6000, 13)
(6000,)
```

```
[16]: x.columns.to_list()
```

```
[16]: ['diameter_mm',
      'material',
      'installation_year',
      'age_years',
      'length_m',
      'operating_pressure_m',
      'soil_type',
      'elevation_m',
      'previous_failures',
      'leakage_incidents',
      'annual_rainfall_mm',
      'recent_rainfall_mm',
      'maintenance_count_24m']
```

```
[21]: #SPLIT THE DATA INTO TRAINING AND TETSING SETS BEFORE SCALING
      from sklearn.model_selection import train_test_split
      x_train, x_test, y_train, y_test = train_test_split(
          x,
          y,
          test_size=0.2,
          random_state=42,
          stratify=y
      )
```

```
[22]: print('Training set:', x_train.shape)
      print('Testing set:', x_test.shape)
```

```
Training set: (4800, 13)
Testing set: (1200, 13)
```

```
[23]: #IDENTIFY CATEGORICAL COLUMNS/VARIABLES IN X COLUMNS
      x.select_dtypes(include='object').columns
```

```
[23]: Index(['material', 'soil_type'], dtype='object')
```

```
[24]: # IDENTIFY NUMERICAL COLUMNS/VARIABLES IN X COLUMNS
x.select_dtypes(include=['int64', 'float64']).columns
```

```
[24]: Index(['diameter_mm', 'installation_year', 'age_years', 'length_m',
          'operating_pressure_m', 'elevation_m', 'previous_failures',
          'leakage_incidents', 'annual_rainfall_mm', 'recent_rainfall_mm',
          'maintenance_count_24m'],
          dtype='object')
```

```
[25]: # SCALE/STANDARDIZE THE NUMERICAL COLUMNS
# some numerical columns have values that are of very different ranges.
# without scaling, a variable with large numerical values such as length_m
# could dominate the distance calculation

from sklearn.preprocessing import StandardScaler
# define the numerical columns
numerical_features = x.select_dtypes(include=['int64', 'float64']).columns

print(numerical_features)
```

```
Index(['diameter_mm', 'installation_year', 'age_years', 'length_m',
      'operating_pressure_m', 'elevation_m', 'previous_failures',
      'leakage_incidents', 'annual_rainfall_mm', 'recent_rainfall_mm',
      'maintenance_count_24m'],
      dtype='object')
```

```
[26]: scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train[numerical_features])
```

```
[27]: x_test_scaled = scaler.transform(x_test[numerical_features])
```

```
[28]: #ENCODING CATEGORICAL VARIABLES/COLUMNS
from sklearn.preprocessing import OneHotEncoder

categorical_features = x.select_dtypes(include='object').columns
print(categorical_features)
```

```
Index(['material', 'soil_type'], dtype='object')
```

```
[29]: #CREATE THE ENCODER
encoder = OneHotEncoder(handle_unknown='ignore')
```

```
[30]: #ENCODE THE CATEGORICAL TESTING AND TRAINING DATA
x_train_enc = encoder.fit_transform(x_train[categorical_features])
```

```
[31]: x_test_enc = encoder.transform(x_test[categorical_features])
```

```
[34]: #COMBINE THE SACLED AND ENCODED DATASETS
```

```
from scipy.sparse import hstack
```

```
x_train_processed = hstack([
    x_train_scaled,
    x_train_enc
])
```

```
x_test_processed = hstack([
    x_test_scaled,
    x_test_enc
])
```

```
[35]: #IMPORT LOGISTIC REGRESSION
```

```
from sklearn.linear_model import LogisticRegression
```

```
[36]: #CREATE THE MODEL
```

```
log_reg = LogisticRegression(random_state=42)
```

```
[37]: #TRAIN THE MODEL
```

```
log_reg.fit(x_train_processed, y_train)
```

```
[37]: LogisticRegression(random_state=42)
```

```
[38]: #MAKE PREDICTIONS
```

```
y_pred_log = log_reg.predict(x_test_processed)
```

```
[39]: #CALCULATE ACCURACY
```

```
from sklearn.metrics import accuracy_score
log_accuracy = accuracy_score(y_test, y_pred_log)
print(f"Logistic Regression Accuracy: {log_accuracy:.4f}")
```

Logistic Regression Accuracy: 0.7658

```
[41]: #CLASSIFICATION REPORT
```

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred_log))
```

	precision	recall	f1-score	support
0	0.78	0.94	0.86	885
1	0.62	0.27	0.38	315
accuracy			0.77	1200
macro avg	0.70	0.61	0.62	1200
weighted avg	0.74	0.77	0.73	1200

```
[42]: #CONFUSION MATRIX  
      from sklearn.metrics import confusion_matrix  
      log_cm = confusion_matrix(y_test, y_pred_log)  
      print(log_cm)
```

```
[[833  52]  
 [229  86]]
```

```
[ ]:
```